

Technical Design Document

AIKosh Dataset Quality Evaluation Toolkit

Document Version: 1.0

Status: Draft

Last Updated: June 18, 2026

Prepared For: AIKosh / IndiaAI Mission

References: PRD_AIKosh_Dataset_Quality_Toolkit v1.0

Classification: Internal Working Document

Table of Contents

1. [Document Purpose](#)
2. [System Overview](#)
3. [Technology Stack — Final Decisions with Justification](#)
4. [Repository & Project Structure](#)
5. [Component Architecture](#)
 - 5.1 FastAPI Application Layer
 - 5.2 Async Job Pipeline (Celery + Redis)
 - 5.3 Dataset Profiling Engine
 - 5.4 Quality Assessment Engine — 15 Domain Modules
 - 5.5 CQI & PRS Engines
 - 5.6 Release Classification Engine
 - 5.7 Report Generation Module
 - 5.8 AIKosh Integration Layer
 - 5.9 Dashboard (React Frontend)
6. [Database Design — Full Schema](#)
 - 6.1 PostgreSQL Schema (DDL)
 - 6.2 Table Descriptions & Indexes
 - 6.3 Migrations Strategy (Alembic)
7. [Object Storage Design](#)
8. [File Ingestion Pipeline — Detailed Design](#)
9. [Domain Scorer Architecture — Code Structure](#)

- 10.[CQI Engine — Implementation Design](#)
 - 11.[PRS Engine — Implementation Design](#)
 - 12.[Release Classification Engine — Implementation Design](#)
 - 13.[Report Generation — Implementation Design](#)
 - 14.[AIKosh Webhook Integration — Implementation Design](#)
 - 15.[Async Job Pipeline — Detailed Design](#)
 - 16.[API Layer — Endpoint Design](#)
 - 17.[Authentication & Authorisation](#)
 - 18.[Audit Log Design](#)
 - 19.[Error Handling & Retry Strategy](#)
 - 20.[Security Design](#)
 - 21.[Configuration & Environment Variables](#)
 - 22.[Deployment Architecture](#)
 - 22.1 Docker Compose (Development)
 - 22.2 Kubernetes (Production)
 - 23.[Scalability Design](#)
 - 24.[Monitoring & Observability](#)
 - 25.[Domain Scorer Configuration \(YAML-driven\)](#)
 - 26.[Inter-Service Communication Map](#)
 - 27.[Key Design Decisions & Rationale](#)
 - 28.[Dependency Versions \(Pinned\)](#)
-

1. Document Purpose

This Technical Design Document (TDD) defines the implementation architecture for the AIKosh Dataset Quality Evaluation Toolkit. It translates the requirements specified in the PRD into concrete technical decisions: exact technology choices with justification, full database schema DDL, directory structure, component interfaces, async job flow, deployment topology, and all design decisions a developer needs to begin coding without ambiguity.

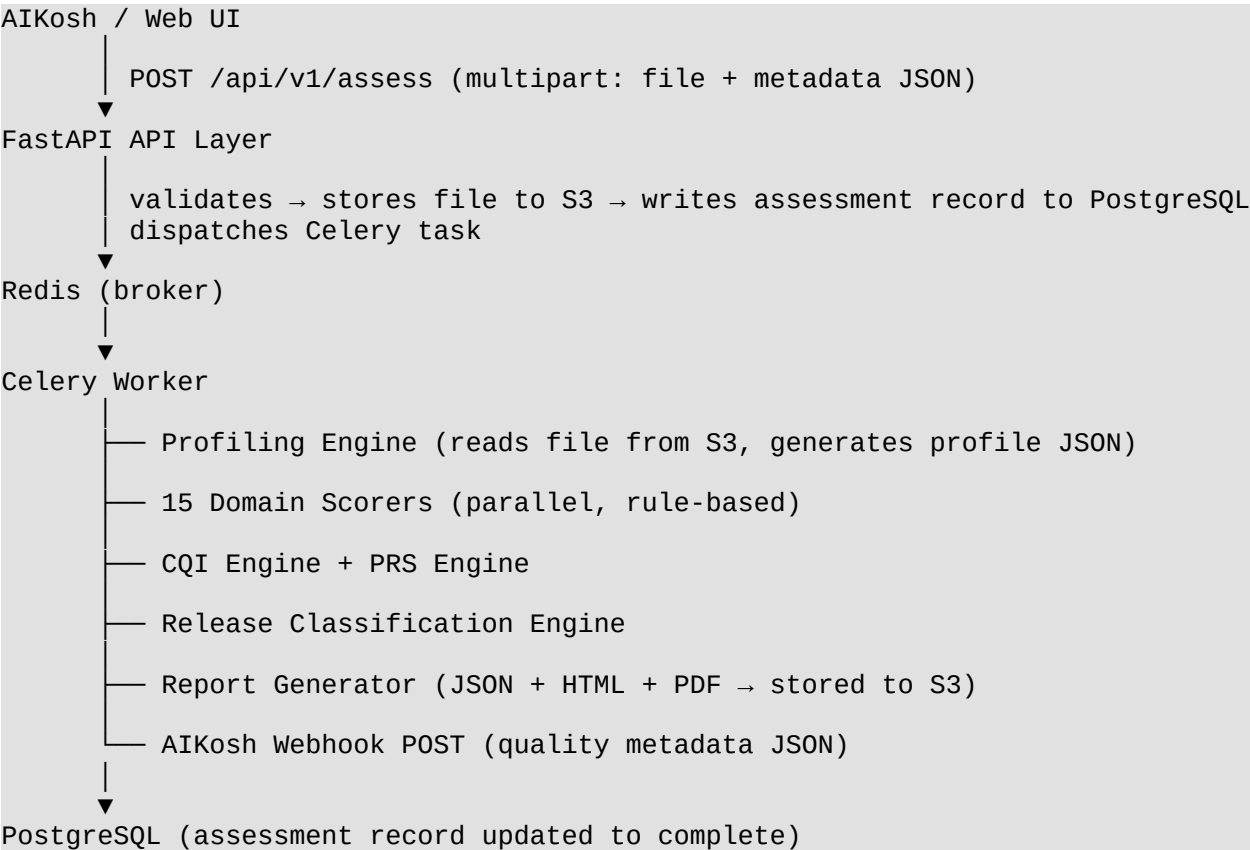
This document is the primary reference for:

- Backend developers implementing the scoring engine, API, and job pipeline
- Frontend developers building the React dashboard
- DevOps engineers managing deployment and infrastructure
- Anyone reviewing technical decisions for the project

2. System Overview

The toolkit is a standalone backend service with a React frontend. It integrates into AIKosh as an external quality-scoring microservice. It is not embedded inside the AIKosh codebase — it runs as a separate deployed service that AIKosh calls via REST API.

Runtime flow (summary):



3. Technology Stack — Final Decisions with Justification

Layer	Technology	Version	Justification
API Framework	FastAPI	0.111.x	Async-native, auto-generates OpenAPI spec, Pydantic validation built-in, fastest Python web framework
ASGI Server	Uvicorn	0.29.x	Production ASGI server for FastAPI; supports multiple workers
Task Queue	Celery	5.4.x	Battle-tested, supports parallel task execution,

Layer	Technology	Version	Justification
			retry logic, priority queues; well-documented FastAPI integration
Message Broker	Redis	7.2.x	Used as both Celery broker AND result backend; simpler than RabbitMQ for this scale; in-memory speed
ORM	SQLAlchemy	2.0.x	Async ORM support in 2.0; Alembic for migrations; industry standard for Python
Database	PostgreSQL	16.x	JSONB support for evidence/gaps fields; reliable ACID compliance for audit logs; strong indexing
Migrations	Alembic	1.13.x	Native SQLAlchemy migration tool; version-controlled schema changes
Dataset Profiling	pandas	2.2.x + ydata-profiling 4.x	pandas for core data ops; ydata-profiling for automated profiling; NumPy for numeric ops
Report Generation (HTML)	Jinja2	3.1.x	Template-based HTML report generation; fast and well-maintained
Report Generation (PDF)	WeasyPrint	62.x	HTML-to-PDF conversion; pure Python, no wkhtmltopdf binary dependency
Object Storage	MinIO (dev) / AWS S3-compatible (prod)	—	S3-compatible API; dataset files and reports stored here, not in DB
Frontend	React	18.x	Component-based; supports charting libraries needed for radar/gauge visualisations
Charting	Recharts	2.x	React-native charting; radar chart + line chart + gauge support

Layer	Technology	Version	Justification
Containerisation	Docker	26.x	All services containerised; consistent environments
Orchestration	Kubernetes	1.30.x	Production deployment; horizontal scaling of Celery workers
CI/CD	GitHub Actions	—	Automated testing, linting, Docker build, deployment
Monitoring	Flower + Prometheus + Grafana	—	Flower for Celery task monitoring; Prometheus + Grafana for system metrics
Logging	structlog	24.x	Structured JSON logging; compatible with log aggregation tools
Config Management	Pydantic Settings	2.x	Environment-variable-based config with type validation

4. Repository & Project Structure

```

aikosh-quality-toolkit/
├── backend/
│   ├── app/
│   │   ├── main.py           # FastAPI app entrypoint
│   │   ├── config.py        # Pydantic Settings config
│   │   └── database.py       # SQLAlchemy engine + session
│   ├── api/
│   │   ├── __init__.py
│   │   └── deps.py          # Shared dependencies (auth, DB
│   │                       session)
│   └── v1/
│       ├── __init__.py
│       ├── assess.py        # Assessment endpoints
│       ├── reports.py       # Report download endpoints
│       ├── datasets.py      # Dataset assessment history
│       └── health.py        # Health check endpoint
├── endpoints
└── models/                  # SQLAlchemy ORM models
    ├── __init__.py
    ├── assessment.py
    ├── domain_score.py
    └── assessment_result.py

```

```

├── dataset_metadata.py
├── audit_log.py
├── schemas/                                # Pydantic request/response schemas
│   ├── __init__.py
│   ├── assessment.py
│   ├── metadata_form.py
│   ├── domain_score.py
│   └── report.py
├── worker/
│   ├── __init__.py
│   ├── celery_app.py                      # Celery app + config
│   └── tasks.py                          # Celery task definitions
├── engine/
│   ├── __init__.py
│   ├── ingestion/
│   │   ├── __init__.py
│   │   └── validator.py                  # File format + size + encoding
├── validation
│   └── parser.py                         # CSV/JSON/XLSX/Parquet/FHIR/ZIP
├── parsers
│   ├── profiler/
│   │   ├── __init__.py
│   │   ├── profiler.py                  # Main profiling orchestrator
│   │   ├── completeness.py             # Column-level completeness analysis
│   │   ├── identifier_scan.py           # PII heuristic scanner
│   │   ├── schema_analysis.py           # Schema inference + consistency
│   │   └── statistical.py               # Numeric distributions, outliers
│   ├── domains/
│   │   ├── __init__.py
│   │   ├── base.py                     # Abstract BaseDomainScorer class
│   │   ├── domain_01_annotation.py
│   │   ├── domain_02_metadata.py
│   │   ├── domain_03_documentation.py
│   │   ├── domain_04_representativeness.py
│   │   ├── domain_05_interoperability.py
│   │   ├── domain_06_ai_readiness.py
│   │   ├── domain_07_privacy.py
│   │   ├── domain_08_security.py
│   │   ├── domain_09_provenance.py
│   │   ├── domain_10_ethics.py
│   │   ├── domain_11_synthetic.py
│   │   ├── domain_12_stewardship.py
│   │   ├── domain_13_model_linkage.py
│   │   ├── domain_14_sustainability.py
│   │   └── domain_15_curation.py
│   ├── scoring/
│   │   ├── __init__.py
│   │   ├── cqi.py                      # CQI computation + band assignment
│   │   ├── prs.py                      # PRS computation + band assignment
│   │   └── release_classifier.py        # CQI × PRS release matrix
│   └── orchestrator.py                  # Coordinates all engine components
├── reports/
│   ├── __init__.py
│   ├── generator.py                    # Report orchestration
│   └── templates/
│       └── quality_report.html          # Jinja2 HTML report template

```

```

├── pdf_renderer.py                # WeasyPrint HTML → PDF
├── integration/
│   ├── __init__.py
│   └── aikosh_webhook.py        # Webhook POST to AIKosh
├── storage/
│   ├── __init__.py
│   └── s3_client.py            # S3/MinIO client wrapper
├── audit/
│   ├── __init__.py
│   └── logger.py               # Audit event writer
├── alembic/                     # DB migration scripts
│   ├── env.py
│   ├── script.py.mako
│   └── versions/
├── config/
│   └── domain_criteria.yaml     # YAML-driven scoring criteria per
domain
├── tests/
│   ├── unit/
│   │   ├── test_domains/      # One test file per domain scorer
│   │   ├── test_cqi.py
│   │   ├── test_prs.py
│   │   └── test_release_classifier.py
│   ├── integration/
│   │   ├── test_assessment_flow.py
│   │   └── test_webhook.py
│   └── fixtures/
│       └── datasets/          # Golden datasets for regression
testing
├── Dockerfile
├── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── RadarChart.jsx
│   │   │   ├── CQIGauge.jsx
│   │   │   ├── PRSGauge.jsx
│   │   │   ├── ReleaseClassBadge.jsx
│   │   │   ├── DomainScoreTable.jsx
│   │   │   ├── GapPanel.jsx
│   │   │   └── ScoreHistory.jsx
│   │   ├── pages/
│   │   │   ├── UploadPage.jsx
│   │   │   ├── DashboardPage.jsx
│   │   │   └── ReportPage.jsx
│   │   └── api/
│   │       └── client.js      # API client
│   └── Dockerfile
├── docker-compose.yml          # Development environment
├── docker-compose.prod.yml
├── k8s/                        # Kubernetes manifests
│   ├── api-deployment.yaml
│   ├── worker-deployment.yaml
│   ├── postgres-statefulset.yaml
│   └── redis-deployment.yaml

```

```
├── ingress.yaml
└── docs/
    ├── PRD_AIKosh_Dataset_Quality_Toolkit.md
    └── TDD_AIKosh_Dataset_Quality_Toolkit.md (this document)
```

5. Component Architecture

5.1 FastAPI Application Layer

The FastAPI app is the entry point for all HTTP traffic. It handles:

- Request validation via Pydantic schemas
- File upload receipt (multipart/form-data)
- Assessment job dispatch to Celery
- Status polling responses
- Report download serving
- API key authentication

The app does **no heavy computation** — it immediately dispatches work to Celery and returns. This keeps API response times under 2 seconds for all submission endpoints.

FastAPI app configuration:

```
# app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api.v1 import assess, reports, datasets, health

app = FastAPI(
    title="AIKosh Dataset Quality Toolkit",
    version="1.0.0",
    docs_url="/docs",
    redoc_url="/redoc",
    openapi_url="/openapi.json"
)

app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

app.include_router(health.router, prefix="/api/v1", tags=["health"])
app.include_router(assess.router, prefix="/api/v1", tags=["assessment"])
app.include_router(reports.router, prefix="/api/v1", tags=["reports"])
app.include_router(datasets.router, prefix="/api/v1", tags=["datasets"])
```

5.2 Async Job Pipeline (Celery + Redis)

Assessment jobs are long-running (up to 3 minutes for 1GB files). FastAPI dispatches them to Celery immediately and returns a job ID. The client either polls or receives a webhook callback on completion.

Celery configuration:

```
# app/worker/celery_app.py
from celery import Celery
from app.config import settings

celery_app = Celery(
    "toolkit",
    broker=settings.REDIS_URL,
    backend=settings.REDIS_URL,
    include=["app.worker.tasks"]
)

celery_app.conf.update(
    task_serializer="json",
    result_serializer="json",
    accept_content=["json"],
    timezone="Asia/Kolkata",
    enable_utc=True,
    task_track_started=True,
    task_acks_late=True,          # Task only acked after completion → prevents
data loss
    worker_prefetch_multiplier=1, # One task per worker at a time for memory
safety
    task_routes={
        "app.worker.tasks.run_assessment": {"queue": "assessment"},
        "app.worker.tasks.send_webhook": {"queue": "webhook"},
    },
    task_soft_time_limit=300,    # 5 min soft limit (sends signal)
    task_time_limit=360,        # 6 min hard limit (kills worker)
)
```

Two queues:

- **assessment** — heavy processing queue; workers here have more memory allocated
 - **webhook** — lightweight outbound HTTP queue; separate workers, never blocked by heavy jobs
-

5.3 Dataset Profiling Engine

The profiling engine reads the uploaded file from S3, generates a Dataset Profile JSON, and stores it as an intermediate artefact referenced by all 15 domain scorers.

Profile JSON structure (full schema):

```
{
  "file": {
    "filename": "tb_cohort_2023.csv",
    "format": "csv",
    "size_bytes": 45231890,
    "sha256": "a3f7c2...",
    "encoding": "UTF-8",
    "compressed_size_bytes": 8102450
  },
  "shape": {
    "rows": 12450,
    "columns": 38
  },
  "columns": [
```

```

{
  "name": "age",
  "dtype": "numeric",
  "completeness_pct": 99.8,
  "missing_count": 25,
  "min": 1,
  "max": 89,
  "mean": 34.2,
  "median": 31.0,
  "std": 14.7,
  "outlier_pct": 0.2,
  "range_violation": false
},
{
  "name": "sex",
  "dtype": "categorical",
  "completeness_pct": 100.0,
  "missing_count": 0,
  "unique_values": 2,
  "value_counts": {"M": 7890, "F": 4560}
},
{
  "name": "patient_name",
  "dtype": "string",
  "completeness_pct": 100.0,
  "pii_flag": "name_pattern",
  "pii_confidence": "High"
}
],
"pii_scan": {
  "direct_identifiers_detected": true,
  "name_columns": ["patient_name"],
  "phone_columns": [],
  "id_columns": ["national_id"],
  "gps_columns": [],
  "dob_columns": ["date_of_birth"]
},
"completeness": {
  "overall_pct": 94.2,
  "columns_below_90pct": ["income_group", "district_code"],
  "columns_below_50pct": []
},
"duplicates": {
  "exact_duplicate_rows": 0,
  "exact_duplicate_pct": 0.0
},
"standards_detected": {
  "icd_codes_present": true,
  "icd_columns": ["diagnosis_icd10"],
  "snomed_codes_present": false,
  "loinc_codes_present": false,
  "fhir_structure": false
},
"split_columns": {
  "split_column_detected": false,
  "fold_column_detected": false
},
"label_columns": {
  "binary_label_detected": true,
  "label_column": "treatment_outcome",
  "class_distribution": {"success": 9870, "failure": 2580},
  "imbalance_ratio": 3.82
},
"age_distribution": {

```

```

    "min": 1, "max": 89,
    "under_18_pct": 8.2,
    "18_to_60_pct": 78.4,
    "over_60_pct": 13.4
  },
  "schema_consistency": {
    "conformant_rows_pct": 99.6,
    "schema_violations": 50
  }
}

```

Profiling engine components:

Component	File	Responsibility
Main orchestrator	profiler/profiler.py	Reads file, dispatches to sub-components, assembles final Profile JSON
Completeness analyser	profiler/completeness.py	Per-column missing value %, overall completeness
PII scanner	profiler/identifier_scan.py	Heuristic detection of names, phones, IDs, GPS, DOB
Schema analyser	profiler/schema_analysis.py	Type inference, ICD/SNOMED/LOINC pattern detection, FHIR structure detection
Statistical analyser	profiler/statistical.py	Distribution stats, outlier detection (IQR), range violation checks

Supported format parsers (engine/ingestion/parser.py):

Format	Parser	Notes
CSV / TSV	pandas.read_csv()	Encoding auto-detected via chardet
XLSX	pandas.read_excel()	First sheet only by default
JSON	pandas.read_json()	Records-oriented or FHIR bundle detected by schema
Parquet	pandas.read_parquet()	PyArrow backend
ZIP	zipfile → recurse	Extracts, detects primary data file, profiles it
FHIR JSON	Custom parser	Detects FHIR Bundle resourceType, flattens to tabular
DICOM manifest	CSV/JSON parser	Treated as tabular manifest

5.4 Quality Assessment Engine — 15 Domain Modules

Each of the 15 domain scorers is a Python class inheriting from `BaseDomainScorer`. They all accept the same inputs and produce the same output shape. They run in **parallel** using Celery's group primitive.

Base class contract (`engine/domains/base.py`):

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional

@dataclass
class DomainScoreResult:
    domain_number: int
    domain_name: str
    score: int  # 0-4
    rationale: str  # Human-readable explanation of the score
    evidence_items: list[str]  # What was found that supported this score
    gaps: list[str]  # What was missing that prevented a higher score
    confidence: str  # "High" | "Medium" | "Low"
    not_applicable: bool = False  # For Domain 11 when no synthetic data

class BaseDomainScorer(ABC):
    def __init__(self, profile: dict, metadata: dict, criteria: dict):
        self.profile = profile  # Dataset Profile JSON from profiler
        self.metadata = metadata  # Metadata intake form values
        self.criteria = criteria  # Loaded from domain_criteria.yaml

    @abstractmethod
    def score(self) -> DomainScoreResult:
        """Score this domain. Must return a DomainScoreResult."""
        pass

    def _determine_confidence(self, data_signals: int, meta_signals: int) -> str:
        """High if majority from DATA, Low if entirely from META."""
        if data_signals >= meta_signals:
            return "High"
        if meta_signals > 0 and data_signals > 0:
            return "Medium"
        return "Low"
```

Domain scorer registration (`engine/domains/__init__.py`):

```
from .domain_01_annotation import AnnotationReliabilityScorer
from .domain_02_metadata import MetadataCompletenessScorer
# ... all 15

DOMAIN_SCORERS = [
    AnnotationReliabilityScorer,
    MetadataCompletenessScorer,
    DocumentationScorer,
    RepresentativenessScorer,
    InteroperabilityScorer,
    AIReadinessScorer,
    PrivacyScorer,
    SecurityScorer,
    ProvenanceScorer,
    EthicsScorer,
    SyntheticDataScorer,
```

```

    StewardshipScorer,
    ModelLinkageScorer,
    SustainabilityScorer,
    CurationScorer,
]

```

Parallel execution in orchestrator:

```

# engine/orchestrator.py
from celery import group
from app.worker.tasks import score_domain

def run_all_domains(assessment_id: str, profile: dict, metadata: dict) -> list:
    """Dispatch all 15 domain scorers in parallel using Celery group."""
    job = group(
        score_domain.s(assessment_id, domain_number, profile, metadata)
        for domain_number in range(1, 16)
    )
    result = job.apply_async(queue="assessment")
    return result.get(timeout=180) # Wait max 3 min for all 15

```

5.5 CQI & PRS Engines

Both are pure Python computation modules — no I/O, no DB calls, just math.

CQI Engine (engine/scoring/cqi.py):

```

from dataclasses import dataclass

CQI_BANDS = [
    (95, "Diamond"),
    (85, "Platinum"),
    (70, "Gold"),
    (50, "Silver"),
    (25, "Bronze"),
    (0, "Remediation"),
]

@dataclass
class CQIResult:
    total_score: int
    max_possible: int
    cqi: float
    band: str
    formula_trace: str
    domain_11_applicable: bool

def compute_cqi(domain_scores: dict[int, int], domain_11_applicable: bool) -> CQIResult:
    scores = {k: v for k, v in domain_scores.items() if v is not None}
    total = sum(scores.values())
    max_possible = 60 if domain_11_applicable else 56
    cqi = round((total / max_possible) * 100, 1)
    band = next(label for threshold, label in CQI_BANDS if cqi >= threshold)
    trace = f"({total} / {max_possible}) × 100 = {cqi}"
    return CQIResult(
        total_score=total,
        max_possible=max_possible,
        cqi=cqi,
        band=band,
        formula_trace=trace,
    )

```

```
        domain_11_applicable=domain_11_applicable
    )
```

PRS Engine (engine/scoring/prs.py):

```
from dataclasses import dataclass

SENSITIVITY_MULTIPLIERS = {
    "standard": 1.0,
    "high_stigma": 1.5,
    "critical": 2.0,
}

PRS_BANDS = [
    (71, "Very High"),
    (41, "High"),
    (16, "Moderate"),
    (0, "Low"),
]

# Step 1 identification risk lookup table (from MIDAS Lite Annexure I)
IDENTIFICATION_RISK_SCORES = {
    "direct_identifiers_present": 50,
    "village_rare_condition": 30,
    "district_or_month": 15,
    "state_or_region": 5,
    "generalised_categories": 5,
}

@dataclass
class PRSResult:
    baseline_risk: float
    sensitivity_class: str
    sensitivity_multiplier: float
    adjusted_risk: float
    prs: int
    band: str
    computation_trace: str
    differential_privacy_applied: bool
    epsilon: Optional[float]

def compute_prs(profile: dict, metadata: dict) -> PRSResult:
    # Step 1: Identification risk
    if profile["pii_scan"]["direct_identifiers_detected"]:
        baseline = 50.0
        basis = "direct_identifiers_present"
    elif metadata.get("differential_privacy_applied"):
        epsilon = metadata["epsilon"]
        baseline = min(100.0, 20.0 * epsilon)
        basis = f"differential_privacy_epsilon_{epsilon}"
    else:
        location = metadata.get("location_granularity", "district")
        rare = metadata.get("rare_condition", False)
        if location == "village" and rare:
            baseline = 30.0
            basis = "village_rare_condition"
        elif location in ("district", "taluk"):
            baseline = 15.0
            basis = "district_or_month"
        elif location in ("state", "region", "national"):
            baseline = 5.0
            basis = "state_or_region"
        else:
```

```

        baseline = 15.0
        basis = "default_moderate"

    # Step 2: Sensitivity multiplier
    sensitivity = metadata.get("sensitivity_class", "standard")
    multiplier = SENSITIVITY_MULTIPLIERS.get(sensitivity, 1.0)
    adjusted = baseline * multiplier
    prs = min(100, round(adjusted))
    band = next(label for threshold, label in PRS_BANDS if prs >= threshold)
    trace = f"baseline={baseline} × multiplier={multiplier} = {adjusted} →
PRS={prs}"

    return PRSResult(
        baseline_risk=baseline,
        sensitivity_class=sensitivity,
        sensitivity_multiplier=multiplier,
        adjusted_risk=adjusted,
        prs=prs,
        band=band,
        computation_trace=trace,

differential_privacy_applied=metadata.get("differential_privacy_applied",
False),
        epsilon=metadata.get("epsilon")
    )

```

5.6 Release Classification Engine

```

# engine/scoring/release_classifier.py
from dataclasses import dataclass

@dataclass
class ReleaseClassification:
    classification: str      # "Open" | "Controlled" | "Restricted"
    justification: str
    policy_override_applied: bool

def classify_release(
    cqi: float,
    prs: int,
    prs_band: str,
    sensitivity_class: str,
    differential_privacy_verified: bool = False
) -> ReleaseClassification:

    # Policy override: high-stigma / critical data
    if sensitivity_class in ("high_stigma", "critical"):
        if prs_band in ("High", "Very High"):
            return ReleaseClassification(
                classification="Restricted",
                justification=f"High-stigma data with {prs_band} PRS. Restricted
per MIDAS policy.",
                policy_override_applied=True
            )
        if not differential_privacy_verified:
            return ReleaseClassification(
                classification="Controlled",
                justification=f"High-stigma data defaults to Controlled unless
Differential Privacy is independently verified.",
                policy_override_applied=True
            )

```

```

# Standard CQI × PRS matrix
if cqi >= 70 and prs_band == "Low":
    return ReleaseClassification("Open", f"CQI={cqi} (≥70) and PRS={prs}
(Low). Open access permitted.", False)
if cqi >= 70 and prs_band == "Moderate":
    return ReleaseClassification("Controlled", f"CQI={cqi} (≥70) but
PRS={prs} (Moderate). Controlled access required.", False)
if prs_band in ("High", "Very High"):
    return ReleaseClassification("Restricted", f"PRS={prs} ({prs_band}).
Restricted regardless of CQI.", False)
if cqi < 50:
    if prs_band == "Low":
        return ReleaseClassification("Controlled", f"CQI={cqi} (<50).
Controlled until quality improved.", False)
        return ReleaseClassification("Restricted", f"CQI={cqi} (<50) and
PRS={prs} ({prs_band}). Restricted.", False)

# Default (CQI 50–69, PRS Low/Moderate)
return ReleaseClassification("Controlled", f"CQI={cqi} (Silver band).
Controlled access.", False)

```

5.7 Report Generation Module

Reports are generated in two passes:

1. **JSON report** — assembled directly from scoring results; stored to S3
2. **HTML report** — Jinja2 renders the JSON data into the report template; stored to S3
3. **PDF** — WeasyPrint converts the HTML to PDF; stored to S3

```

# reports/generator.py
from jinja2 import Environment, FileSystemLoader
from weasyprint import HTML
from app.storage.s3_client import upload_to_s3
import json

TEMPLATES_DIR = "app/reports/templates"

def generate_report(assessment_result: dict, assessment_id: str) -> dict:
    """Returns URLs for JSON, HTML, and PDF reports."""

    # 1. JSON report
    json_bytes = json.dumps(assessment_result, indent=2).encode("utf-8")
    json_url = upload_to_s3(json_bytes, f"reports/{assessment_id}/report.json",
"application/json")

    # 2. HTML report via Jinja2
    env = Environment(loader=FileSystemLoader(TEMPLATES_DIR))
    template = env.get_template("quality_report.html")
    html_content = template.render(**assessment_result)
    html_bytes = html_content.encode("utf-8")
    html_url = upload_to_s3(html_bytes, f"reports/{assessment_id}/report.html",
"text/html")

    # 3. PDF via WeasyPrint
    pdf_bytes = HTML(string=html_content).write_pdf()
    pdf_url = upload_to_s3(pdf_bytes, f"reports/{assessment_id}/report.pdf",
"application/pdf")

```



```
return {"json_url": json_url, "html_url": html_url, "pdf_url": pdf_url}
```

5.8 AIKosh Integration Layer

The toolkit POSTs quality metadata to an AIKosh-provided webhook URL upon assessment completion. This is dispatched as a separate Celery task in the `webhook` queue.

```
# integration/aikosh_webhook.py
import httpx
from app.config import settings
from app.audit.logger import audit_event

async def post_to_aikosh(assessment_id: str, dataset_id: str, quality_payload: dict):
    webhook_url = settings.AIKOSH_WEBHOOK_URL
    headers = {
        "Authorization": f"Bearer {settings.AIKOSH_WEBHOOK_SECRET}",
        "Content-Type": "application/json",
        "X-Toolkit-Version": "1.0.0",
        "X-Assessment-ID": assessment_id,
    }
    async with httpx.AsyncClient(timeout=30.0) as client:
        response = await client.post(webhook_url, json=quality_payload,
headers=headers)
        response.raise_for_status()
        audit_event(assessment_id, "aikosh_webhook_sent", {"status_code":
response.status_code})
```

Retry policy: If AIKosh webhook fails, Celery retries 3× with exponential backoff (30s, 120s, 480s).

5.9 Dashboard (React Frontend)

The React dashboard is a separate single-page application served from a CDN or Nginx container. It calls the toolkit's REST API directly.

Key components:

Component	Library	Purpose
<code>RadarChart.jsx</code>	Recharts <code>RadarChart</code>	15-domain score spider chart
<code>CQIGauge.jsx</code>	Recharts <code>RadialBarChart</code>	0–100 CQI gauge with band colour
<code>PRSGauge.jsx</code>	Recharts <code>RadialBarChart</code>	0–100 PRS gauge with risk colour
<code>ReleaseClassBadge.jsx</code>	Custom CSS	Green/Amber/Red badge
<code>DomainScoreTable.jsx</code>	HTML table	All 15 domains, score, confidence, rationale
<code>GapPanel.jsx</code>	Sorted list	Domains scoring ≤ 2 , sorted ascending
<code>ScoreHistory.jsx</code>	Recharts <code>LineChart</code>	CQI/PRS trend over re-

Component	Library	Purpose
		assessments

Band colour scheme:

Band	CQI	Colour
Diamond	≥95	#4FC3F7 (cyan)
Platinum	85–94	#B0BEC5 (silver)
Gold	70–84	#FFD54F (amber-gold)
Silver	50–69	#90A4AE (blue-grey)
Bronze	25–49	#A1887F (brown)
Remediation	<25	#EF5350 (red)

6. Database Design — Full Schema

6.1 PostgreSQL Schema (DDL)

```
-- =====
-- AIKosh Quality Toolkit – PostgreSQL Schema
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

-- -----
-- ENUM types
-- -----
CREATE TYPE assessment_status AS ENUM ('queued', 'processing', 'complete',
'failed');
CREATE TYPE release_class AS ENUM ('Open', 'Controlled', 'Restricted');
CREATE TYPE confidence_level AS ENUM ('High', 'Medium', 'Low');
CREATE TYPE sensitivity_class AS ENUM ('standard', 'high_stigma', 'critical');

-- -----
-- TABLE: assessments
-- Main assessment lifecycle record
-- -----
CREATE TABLE assessments (
  assessment_id      UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  dataset_id        VARCHAR(255) NOT NULL,          -- AIKosh dataset ID
  submitter_id      VARCHAR(255),                  -- API key owner /
user ID
  submission_timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completion_timestamp TIMESTAMPTZ,
  status            assessment_status NOT NULL DEFAULT 'queued',
  toolkit_version   VARCHAR(20) NOT NULL DEFAULT '1.0.0',
  domain_11_applicable BOOLEAN NOT NULL DEFAULT TRUE,
  assessment_mode    VARCHAR(20) NOT NULL DEFAULT 'full', -- 'lite' |
'full'
  file_format       VARCHAR(50),
  file_size_bytes   BIGINT,
  file_sha256       VARCHAR(64),
```

```

        s3_file_key          VARCHAR(500),                -- Temp S3 key for
dataset file
        error_message        TEXT,                        -- Populated on
failure
        celery_task_id       VARCHAR(255),
        created_at           TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at           TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_assessments_dataset_id ON assessments(dataset_id);
CREATE INDEX idx_assessments_status ON assessments(status);
CREATE INDEX idx_assessments_submission_timestamp ON
assessments(submission_timestamp DESC);

-----
-- TABLE: dataset_metadata
-- Metadata intake form values submitted alongside the file
-----
CREATE TABLE dataset_metadata (
    metadata_id             UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    assessment_id           UUID NOT NULL REFERENCES assessments(assessment_id) ON
DELETE CASCADE,
    dataset_name            VARCHAR(500) NOT NULL,
    dataset_version         VARCHAR(100),
    dataset_type            VARCHAR(50),                --
tabular/imaging/text/multimodal
    study_type             VARCHAR(100),                -- RCT/cohort/cross-
sectional/registry
    target_population       TEXT,
    geographic_coverage     VARCHAR(255),
    age_range_min          INT,
    age_range_max          INT,
    sex_distribution        VARCHAR(50),
    num_sites              INT,
    collection_start_date   DATE,
    collection_end_date     DATE,
    annotation_methodology TEXT,
    num_annotators         INT,
    irr_method             VARCHAR(100),
    irr_value              DECIMAL(5,3),
    standards_used         VARCHAR(255),
    ethics_approval_ref     VARCHAR(255),
    consent_type           VARCHAR(100),
    deidentification_method VARCHAR(255),
    differential_privacy_applied BOOLEAN DEFAULT FALSE,
    dp_epsilon             DECIMAL(10,4),
    sensitivity_class       sensitivity_class NOT NULL DEFAULT 'standard',
    persistent_identifier   VARCHAR(500),
    license_type           VARCHAR(100),
    synthetic_data_pct      DECIMAL(5,2),
    access_control_method   VARCHAR(255),
    linked_model_ids        JSONB,
    data_dictionary_uploaded BOOLEAN DEFAULT FALSE,
    provenance_pipeline_available BOOLEAN DEFAULT FALSE,
    github_repo_url        VARCHAR(500),
    changelog_provided     BOOLEAN DEFAULT FALSE,
    version_format         VARCHAR(50),
    raw_form_json          JSONB,                        -- Full form snapshot
for audit
    created_at             TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_dataset_metadata_assessment_id ON
dataset_metadata(assessment_id);

```

```

-----
-- TABLE: dataset_profiles
-- Profiler output – stored for audit and domain scorer reference
-----
CREATE TABLE dataset_profiles (
    profile_id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    assessment_id       UUID NOT NULL REFERENCES assessments(assessment_id) ON
DELETE CASCADE,
    profile_json        JSONB NOT NULL,                -- Full Dataset
    Profile JSON
    profiled_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    profiler_version    VARCHAR(20) NOT NULL DEFAULT '1.0.0'
);

CREATE INDEX idx_dataset_profiles_assessment_id ON
dataset_profiles(assessment_id);

-----
-- TABLE: domain_scores
-- One row per domain per assessment (15 rows per assessment)
-----
CREATE TABLE domain_scores (
    score_id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    assessment_id       UUID NOT NULL REFERENCES assessments(assessment_id) ON
DELETE CASCADE,
    domain_number       SMALLINT NOT NULL CHECK (domain_number BETWEEN 1 AND
15),
    domain_name         VARCHAR(100) NOT NULL,
    score              SMALLINT CHECK (score BETWEEN 0 AND 4),
    not_applicable      BOOLEAN NOT NULL DEFAULT FALSE,
    rationale           TEXT,
    evidence_items      JSONB,                        -- Array of evidence
strings
    gaps               JSONB,                        -- Array of gap
strings
    confidence_level    confidence_level,
    data_signals_count  SMALLINT,                    -- How many checks
came from DATA
    meta_signals_count  SMALLINT,                    -- How many checks
came from META
    scored_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    UNIQUE (assessment_id, domain_number)
);

CREATE INDEX idx_domain_scores_assessment_id ON domain_scores(assessment_id);
CREATE INDEX idx_domain_scores_domain_number ON domain_scores(domain_number);

-----
-- TABLE: assessment_results
-- Final computed CQI, PRS, and release classification
-----
CREATE TABLE assessment_results (
    result_id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    assessment_id       UUID NOT NULL REFERENCES
assessments(assessment_id) ON DELETE CASCADE,
    cqi                DECIMAL(5,1) NOT NULL,
    cqi_band           VARCHAR(20) NOT NULL,
    total_domain_score SMALLINT NOT NULL,
    max_possible_score SMALLINT NOT NULL,
    cqi_formula_trace  VARCHAR(100),
    prs               SMALLINT NOT NULL CHECK (prs BETWEEN 0 AND
100),

```

```

    prs_band                                VARCHAR(20) NOT NULL,
    prs_baseline_risk                        DECIMAL(6,2),
    prs_sensitivity_multiplier               DECIMAL(4,2),
    prs_computation_trace                    TEXT,
    release_classification                   release_class NOT NULL,
    classification_justification             TEXT,
    policy_override_applied                 BOOLEAN NOT NULL DEFAULT FALSE,
    report_json_url                         VARCHAR(500),
    report_html_url                         VARCHAR(500),
    report_pdf_url                         VARCHAR(500),
    computed_at                             TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    UNIQUE (assessment_id)
);

CREATE INDEX idx_assessment_results_assessment_id ON
assessment_results(assessment_id);
CREATE INDEX idx_assessment_results_release_classification ON
assessment_results(release_classification);
CREATE INDEX idx_assessment_results_cqi ON assessment_results(cqi DESC);

-- -----
-- TABLE: audit_logs
-- Append-only event log for every assessment lifecycle event
-- -----
CREATE TABLE audit_logs (
    log_id                                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    assessment_id                         UUID NOT NULL REFERENCES assessments(assessment_id) ON
DELETE CASCADE,
    event_type                            VARCHAR(100) NOT NULL,
    event_timestamp                       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    component                             VARCHAR(100),                -- Which engine
component fired this
    event_detail                          JSONB,                        -- Arbitrary
structured detail
    severity                              VARCHAR(20) DEFAULT 'INFO'    -- INFO / WARNING /
ERROR
);

CREATE INDEX idx_audit_logs_assessment_id ON audit_logs(assessment_id);
CREATE INDEX idx_audit_logs_event_timestamp ON audit_logs(event_timestamp DESC);
CREATE INDEX idx_audit_logs_event_type ON audit_logs(event_type);

-- Prevent deletion of audit logs (append-only enforcement)
CREATE RULE no_delete_audit AS ON DELETE TO audit_logs DO INSTEAD NOTHING;

-- -----
-- TABLE: api_keys
-- API key management for submitters and AIKosh integration
-- -----
CREATE TABLE api_keys (
    key_id                                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    key_hash                              VARCHAR(64) NOT NULL UNIQUE,    -- SHA-256 hash of key
    key_prefix                            VARCHAR(10) NOT NULL,        -- First 8 chars for
identification
    owner_name                            VARCHAR(255) NOT NULL,
    role                                  VARCHAR(20) NOT NULL DEFAULT 'submitter', --
submitter/reviewer/admin
    is_active                             BOOLEAN NOT NULL DEFAULT TRUE,
    created_at                             TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_used_at                           TIMESTAMPTZ,
    expires_at                             TIMESTAMPTZ
);

```

```
CREATE INDEX idx_api_keys_key_hash ON api_keys(key_hash);
CREATE INDEX idx_api_keys_is_active ON api_keys(is_active);
```

6.2 Table Descriptions & Indexes

Table	Purpose	Key Indexes
assessments	Lifecycle record for every assessment job	dataset_id, status, submission_timestamp
dataset_metadata	Metadata form values for each submission	assessment_id
dataset_profiles	Full profiler output JSON	assessment_id
domain_scores	15 domain scores per assessment	assessment_id, domain_number
assessment_results	Final CQI, PRS, release class	assessment_id, release_classification, cqi
audit_logs	Append-only event log	assessment_id, event_timestamp, event_type
api_keys	API key registry	key_hash, is_active

6.3 Migrations Strategy (Alembic)

- All schema changes go through Alembic migration scripts
- Migration files named: YYYYMMDD_HHMM_<description>.py
- Every migration is reversible (has `downgrade()` function)
- Migrations run automatically on container startup via `alembic upgrade head`
- Never apply raw DDL to production directly

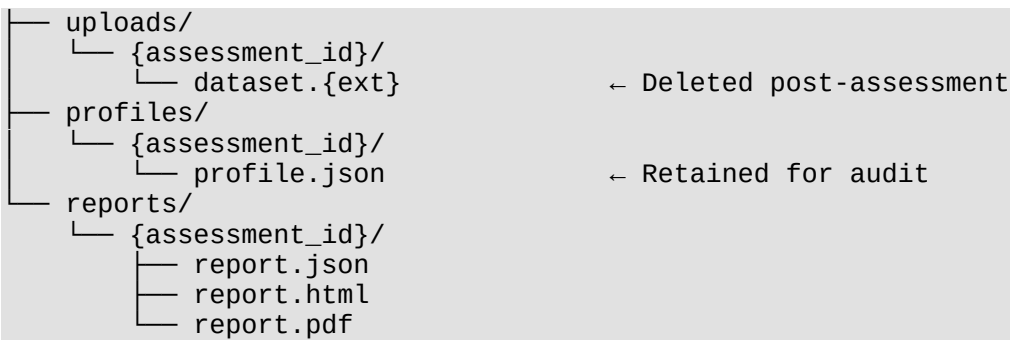
7. Object Storage Design

Object storage (MinIO in dev, S3-compatible in prod) holds:

- Uploaded dataset files (temporary — deleted after assessment)
- Generated reports (JSON, HTML, PDF — retained for 5 years)
- Dataset profile JSONs (retained for audit purposes)

Bucket layout:

```
aikosh-toolkit-bucket/
```



Access policies:

- uploads/ — write on ingestion, read by worker, delete post-assessment
- profiles/ — write by profiler worker, read-only after that
- reports/ — write by report generator, public-read via pre-signed URLs (time-limited)

Pre-signed URLs: All report download links returned to clients are pre-signed S3 URLs with 24-hour expiry. No direct bucket access.

8. File Ingestion Pipeline — Detailed Design

```
POST /api/v1/assess (multipart/form-data)
├── FastAPI receives file + metadata JSON
├── Pydantic validates metadata JSON schema
├── File validation (validator.py):
│   ├── Check MIME type against allowed list
│   ├── Check file size ≤ 5GB
│   ├── Check filename for path traversal (reject ../ etc.)
│   └── Compute SHA-256 hash
├── Store file to S3: uploads/{assessment_id}/dataset.{ext}
├── Write assessments record (status=queued)
├── Write dataset_metadata record
├── Dispatch Celery task: run_assessment.delay(assessment_id)
└── Return HTTP 202 Accepted:
    { "assessment_id": "uuid", "status": "queued" }
```

File size handling for large files (>500MB):

- Profiling uses pandas sampling: random 100,000 row sample for statistical analysis
 - Full scan for: PII detection, ICD code detection, completeness (counts NaN without loading all data)
 - Report clearly states "Profiled on sample" if sampling was applied
-

9. Domain Scorer Architecture — Code Structure

Each domain scorer follows the same pattern. Example for Domain 7 (Privacy):

```
# engine/domains/domain_07_privacy.py
from .base import BaseDomainScorer, DomainScoreResult

class PrivacyScorer(BaseDomainScorer):
    DOMAIN_NUMBER = 7
    DOMAIN_NAME = "Privacy & Identifiability"

    def score(self) -> DomainScoreResult:
        evidence = []
        gaps = []
        data_signals = 0
        meta_signals = 0

        # --- DATA checks ---
        pii = self.profile.get("pii_scan", {})
        direct_ids = pii.get("direct_identifiers_detected", False)

        if direct_ids:
            # Direct identifiers present → Score 0, no further evaluation needed
            gaps.append(f"Direct identifiers detected: {pii.get('name_columns',
[])} {pii.get('id_columns', [])}")
            data_signals += 1
            return DomainScoreResult(
                domain_number=self.DOMAIN_NUMBER,
                domain_name=self.DOMAIN_NAME,
                score=0,
                rationale="Direct identifiers (names, IDs, DOB) detected in
dataset columns.",
                evidence_items=evidence,
                gaps=gaps,
                confidence="High"
            )

        evidence.append("No direct identifiers detected in column scan.")
        data_signals += 1

        # --- META checks ---
        deident_method = self.metadata.get("deidentification_method")
        dp_applied = self.metadata.get("differential_privacy_applied", False)
        dp_epsilon = self.metadata.get("dp_epsilon")
        location = self.metadata.get("geographic_coverage", "district")

        if not deident_method:
            gaps.append("No de-identification method documented.")
            meta_signals += 1
            computed_score = 1
        else:
            evidence.append(f"De-identification method documented:
{deident_method}")
            meta_signals += 1
            if dp_applied and dp_epsilon:
                evidence.append(f"Differential Privacy applied with
ε={dp_epsilon}")
                computed_score = 4
            elif "k-anonymity" in (deident_method or "").lower():
                evidence.append("k-anonymity method declared.")
                computed_score = 4
            else:
                computed_score = 3
```



```
confidence = self._determine_confidence(data_signals, meta_signals)
rationale = f"Score {computed_score}: {'; '.join(evidence[:2])}"

return DomainScoreResult(
    domain_number=self.DOMAIN_NUMBER,
    domain_name=self.DOMAIN_NAME,
    score=computed_score,
    rationale=rationale,
    evidence_items=evidence,
    gaps=gaps,
    confidence=confidence
)
```

10. CQI Engine — Implementation Design

See Section 5.5 for code. Additional notes:

- Domain 11 is excluded from the denominator only if `domain_11_applicable=False` (set when metadata form `synthetic_data_pct` is 0 or null)
- If a domain scorer returns `not_applicable=True`, its score is `None` and excluded from sum
- CQI is always rounded to 1 decimal place
- CQI is recomputed from scratch on every assessment — never cached from a prior version

11. PRS Engine — Implementation Design

See Section 5.5 for code. Additional notes:

- PRS uses profile data (PII scan) for Step 1 identification risk
- Sensitivity class comes entirely from the metadata form (custodian self-declares)
- The toolkit flags `sensitivity_class` as a metadata-form-only field with `confidence=Low`
- PRS is an integer (rounded), not a decimal — consistent with MIDAS specification
- If both direct identifiers are present AND sensitivity is `high_stigma` → $PRS = \min(100, 50 \times 1.5) = 75$ → Very High

12. Release Classification Engine — Implementation Design

See Section 5.6 for code. Additional notes:

- The release matrix is encoded as logic, not a lookup table — this makes it auditable and traceable
- Policy overrides are always recorded in `classification_justification` and `policy_override_applied=True`

- The `differential_privacy_verified` flag is only set to `True` if the metadata form includes a documented DP epsilon AND the dataset passed Domain 7 with score 4
 - Release classification is always deterministic given the same CQI, PRS, and sensitivity class
-

13. Report Generation — Implementation Design

JSON report structure: See PRD Appendix B for the complete schema.

HTML report template (`quality_report.html`): The Jinja2 template renders:

- Header with dataset name, assessment ID, date, toolkit version
- CQI gauge + band badge
- PRS gauge + band badge
- Release classification badge (colour-coded)
- Domain scores table (all 15 rows)
- Strengths section (scores ≥ 3)
- Gaps section (scores ≤ 2)
- PRS computation trace
- CQI formula trace
- Assumptions footnotes
- Audit log reference

PDF generation note: WeasyPrint requires all assets (CSS, fonts) to be bundled with the HTML. The template uses inline CSS with web-safe fonts to ensure PDF renders correctly without external dependencies.

14. AIKosh Webhook Integration — Implementation Design

Webhook payload (identical to PRD Section 17.9 schema):

```
{
  "assessment_id": "uuid",
  "dataset_id": "aikosh_dataset_id",
  "assessed_at": "2026-06-18T10:34:22Z",
  "toolkit_version": "1.0.0",
  "cqi": 74.2,
  "cqi_band": "Gold",
  "prs": 12,
  "prs_band": "Low",
  "release_classification": "Open",
  "domain_scores": { ... },
  "domain_11_applicable": false,
  "report_url": "https://toolkit.aikosh.gov.in/reports/uuid",
  "audit_log_id": "uuid"
}
```

Idempotency: The webhook payload includes the `assessment_id`. If AIKosh receives a duplicate POST for the same `assessment_id`, it should update the record, not create a duplicate. The toolkit marks webhook success in the audit log and does not retry if the response indicates the record already exists (HTTP 200 or 409).

Timeout & failure handling:

- HTTP client timeout: 30 seconds per attempt
- Retry schedule: 3 retries at 30s, 2min, 8min intervals (exponential backoff)
- After 3 failed retries: webhook marked as **failed** in audit log; AIKosh can call GET `/api/v1/assess/{id}` to pull results manually

15. Async Job Pipeline — Detailed Design

FastAPI: POST `/api/v1/assess`

```
├── validates → stores to S3 → writes DB record (status=queued)
└── run_assessment.delay(assessment_id)    [dispatched to Redis]
```

Celery Worker (assessment queue):

`run_assessment(assessment_id)`

```
├── STEP 1: Update status → processing
├── audit_event("assessment_started")
├── STEP 2: Load metadata from DB
├── STEP 3: Download file from S3
├── STEP 4: Run profiler → generate profile JSON
│   ├── → store profile to S3
│   └── → write dataset_profiles record
├── STEP 5: Run 15 domain scorers in parallel (Celery group)
│   ├── → Each scorer returns DomainScoreResult
│   └── → Write 15 domain_scores records to DB
├── STEP 6: Compute CQI
├── STEP 7: Compute PRS
├── STEP 8: Run release classification
├── STEP 9: Generate reports (JSON + HTML + PDF) → upload to S3
├── STEP 10: Write assessment_results record to DB
├── STEP 11: Update assessment status → complete
│   └── → Set completion_timestamp
└── STEP 12: Delete dataset file from S3
    └── → audit_event("dataset_file_deleted")
```

```
— STEP 13: Dispatch webhook task
— STEP 14: audit_event("assessment_complete")
```

Celery Worker (webhook queue):

```
send_webhook(assessment_id)
— Load assessment_results from DB
— Construct webhook payload
— POST to AIKosh webhook URL (with retry)
— audit_event("webhook_sent" or "webhook_failed")
```

On failure at any step:

- Celery marks task as FAILED
 - Assessment status updated to **failed**
 - **error_message** field populated with exception info
 - `audit_event("assessment_failed", {"step": N, "error": "..."})` written
 - No partial results returned to AIKosh
-

16. API Layer — Endpoint Design

POST /api/v1/assess

Purpose: Submit a dataset for assessment.

Auth: Bearer API key required.

Content-Type: multipart/form-data

Request parts:

- **file** (required) — dataset file
- **metadata** (required) — JSON string matching `MetadataFormSchema`
- **webhook_url** (optional) — override AIKosh webhook URL for this submission
- **attachments** (optional, multiple) — data dictionary, SOP, consent docs

Response (HTTP 202):

```
{
  "assessment_id": "a3f7c2d1-9b4e-4f2a-bc31-7d8e9f1a2b3c",
  "status": "queued",
  "estimated_completion_seconds": 180,
  "poll_url": "/api/v1/assess/a3f7c2d1-9b4e-4f2a-bc31-7d8e9f1a2b3c"
}
```

GET /api/v1/assess/{assessment_id}

Purpose: Poll assessment status or get final results.

Auth: Bearer API key required.

Response when processing (HTTP 200):

```
{
  "assessment_id": "...",
  "status": "processing",
  "started_at": "2026-06-18T10:30:00Z"
}
```

Response when complete (HTTP 200):

```
{
  "assessment_id": "...",
  "status": "complete",
  "dataset_id": "aikosh-hrd-00421",
  "assessed_at": "2026-06-18T10:34:22Z",
  "cqi": 74.2,
  "cqi_band": "Gold",
  "prs": 12,
  "prs_band": "Low",
  "release_classification": "Open",
  "domain_scores": { ... },
  "report_urls": {
    "json": "https://...",
    "html": "https://...",
    "pdf": "https://..."
  }
}
```

GET /api/v1/assess/{assessment_id}/report

Purpose: Download full quality report.

Auth: Bearer API key required.

Query param: format — json (default) | html | pdf

Response: Redirects to pre-signed S3 URL for the requested format.

GET /api/v1/assess/{assessment_id}/audit

Purpose: Retrieve full audit log for an assessment.

Auth: Bearer API key, reviewer/admin role required.

Response (HTTP 200):

```
{
  "assessment_id": "...",
  "events": [
    {
      "log_id": "...",
      "event_type": "assessment_started",
      "event_timestamp": "2026-06-18T10:30:00Z",
      "component": "worker",
      "event_detail": {},
    }
  ]
}
```

```
    "severity": "INFO"
  }
]
```

GET /api/v1/datasets/{dataset_id}/assessments

Purpose: List all assessments for a given AIKosh dataset ID.

Auth: Bearer API key required.

Query params: `limit` (default 10, max 50), `offset` (default 0)

GET /api/v1/health

Purpose: Health check for load balancer and monitoring.

Auth: None required.

Response (HTTP 200):

```
{
  "status": "healthy",
  "version": "1.0.0",
  "timestamp": "2026-06-18T10:00:00Z",
  "dependencies": {
    "postgres": "ok",
    "redis": "ok",
    "s3": "ok"
  }
}
```

17. Authentication & Authorisation

Mechanism: API key as Bearer token in `Authorization` header.

`Authorization: Bearer tkt_live_abc123xyz456...`

Key format: `tkt_live_{32 random chars}` for production; `tkt_test_{32 random chars}` for testing.

Storage: Only SHA-256 hash stored in `api_keys` table — raw key never stored after generation.

Roles:

Role	Permissions
submitter	POST /assess, GET /assess/{id}, GET /assess/{id}/report, GET /datasets/{id}/assessments
reviewer	All submitter permissions + GET /assess/{id}/audit
admin	All permissions + key management

FastAPI dependency (api/deps.py):

```
from fastapi import Security, HTTPException
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import hashlib

bearer_scheme = HTTPBearer()

async def get_current_api_key(
    credentials: HTTPAuthorizationCredentials = Security(bearer_scheme),
    db: AsyncSession = Depends(get_db)
) -> APIKey:
    key_hash = hashlib.sha256(credentials.credentials.encode()).hexdigest()
    key = await db.execute(
        select(APIKey).where(APIKey.key_hash == key_hash, APIKey.is_active ==
True)
    )
    if not key:
        raise HTTPException(status_code=401, detail="Invalid or inactive API
key")
    return key
```

18. Audit Log Design

Every significant event in the assessment lifecycle writes an audit log entry. Events are append-only (enforced via PostgreSQL rule — see schema).

Standard event types:

Event Type	Component	When
assessment_submitted	api	File received, job dispatched
file_stored_s3	worker	Dataset file uploaded to S3
profiling_started	profiler	Profiling engine begins
profiling_complete	profiler	Profile JSON generated
domain_scoring_starte d	orchestrator	15 domain scorers dispatched
domain_{N}_scored	domain_{N}_scorer	Each domain scores
cqi_computed	cqi_engine	CQI calculated
prs_computed	prs_engine	PRS calculated
release_classified	release_engine	Release class assigned
report_generated	report_generator	JSON+HTML+PDF stored to S3
dataset_file_deleted	worker	Raw file purged post-assessment
assessment_complete	worker	Assessment marked complete
aikosh_webhook_sent	webhook	Webhook POST successful

Event Type	Component	When
aikosh_webhook_failed	webhook	Webhook POST failed (with retry count)
assessment_failed	worker	Any step failed

Audit log integrity: All audit log entries are cryptographically chained — each entry's `event_detail` includes the SHA-256 hash of the previous entry for this assessment, enabling tamper detection.

19. Error Handling & Retry Strategy

API layer errors:

Condition	HTTP Status	Response
Invalid file format	422	<code>{"error": "unsupported_format", "detail": "..."} </code>
File too large	413	<code>{"error": "file_too_large", "max_bytes": 5368709120} </code>
Invalid metadata JSON	422	Pydantic validation error detail
Invalid API key	401	<code>{"error": "invalid_api_key"} </code>
Assessment not found	404	<code>{"error": "not_found"} </code>
Server error	500	<code>{"error": "internal_error", "assessment_id": "..."} </code>

Worker errors (Celery retry policy):

Step	Retry Count	Backoff
S3 file download	3	5s, 20s, 60s
Domain scorer (individual)	2	10s, 30s
Report generation	2	10s, 30s
AIKosh webhook	3	30s, 120s, 480s

If a domain scorer fails after retries, it scores 0 with `confidence=Low` and a `gaps` entry noting "Scoring failed due to error — defaulted to 0".

20. Security Design

Requirement	Implementation
Encryption at rest	S3 server-side encryption (AES-256); PostgreSQL tablespace encryption
Encryption in transit	TLS 1.3 enforced on all API endpoints via Nginx/Ingress
File isolation	Each dataset processed in its own Celery task with a unique temp directory
File deletion	Dataset file deleted from S3 immediately after report generation (Step 12 in pipeline)
No raw data in DB	Only profile metadata (statistics, counts, PII flags) stored in DB — never raw data
Path traversal prevention	Filename sanitised with <code>secure_filename()</code> on upload
Malicious file prevention	MIME type check + file magic bytes check (not just extension)
API key security	Keys stored as SHA-256 hash only; raw key shown once on creation
Rate limiting	100 requests/minute per API key via Redis-backed rate limiter
CORS	Restricted to AIKosh domain in production
SQL injection prevention	All DB queries via SQLAlchemy parameterised queries; no raw SQL
Dependency scanning	<code>pip-audit</code> run in CI/CD pipeline on every push
DPDP Act compliance	All personally identifiable metadata fields handled per DPDP Act 2023

21. Configuration & Environment Variables

All configuration via environment variables, loaded via Pydantic Settings.

```
# app/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # Database
    DATABASE_URL: str = "postgresql+asyncpg://user:pass@host:5432/db"

    # Redis
    REDIS_URL: str = "redis://host:6379/0"

    # S3 / Object Storage
    S3_ENDPOINT_URL: str = "https://s3.amazonaws.com or"
```

```

MinIO URL
  S3_BUCKET_NAME: str # aikosh-toolkit-bucket
  S3_ACCESS_KEY: str
  S3_SECRET_KEY: str
  S3_REGION: str = "ap-south-1"

  # AIKosh Integration
  AIKOSH_WEBHOOK_URL: str #
https://api.aikosh.gov.in/webhooks/quality
  AIKOSH_WEBHOOK_SECRET: str # Bearer token for AIKosh webhook
auth

  # File Limits
  MAX_FILE_SIZE_BYTES: int = 5_368_709_120 # 5GB
  PROFILING_SAMPLE_ROWS: int = 100_000 # Rows to sample for large
datasets

  # Toolkit
  TOOLKIT_VERSION: str = "1.0.0"
  ENVIRONMENT: str = "production" # development / staging /
production

  # Security
  ALLOWED_ORIGINS: list[str] = ["https://aikosh.indiaai.gov.in"]

  class Config:
      env_file = ".env"

settings = Settings()

```

.env.example:

```

DATABASE_URL=postgresql+asyncpg://toolkit:password@localhost:5432/toolkit_db
REDIS_URL=redis://localhost:6379/0
S3_ENDPOINT_URL=http://localhost:9000
S3_BUCKET_NAME=aikosh-toolkit-dev
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin
AIKOSH_WEBHOOK_URL=https://api.aikosh.gov.in/webhooks/quality
AIKOSH_WEBHOOK_SECRET=your_secret_here
ENVIRONMENT=development

```

22. Deployment Architecture

22.1 Docker Compose (Development)

```

# docker-compose.yml
version: "3.9"

services:
  api:
    build: ./backend
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [postgres, redis, minio]
    volumes: [".:/backend:/app"]

  worker_assessment:
    build: ./backend

```

```

    command: celery -A app.worker.celery_app worker -Q assessment -c 4 --
loglevel=info
    env_file: .env
    depends_on: [postgres, redis, minio]

worker_webhook:
    build: ./backend
    command: celery -A app.worker.celery_app worker -Q webhook -c 2 --
loglevel=info
    env_file: .env
    depends_on: [redis]

flower:
    build: ./backend
    command: celery -A app.worker.celery_app flower --port=5555
    ports: ["5555:5555"]
    depends_on: [redis]

postgres:
    image: postgres:16-alpine
    environment:
        POSTGRES_DB: toolkit_db
        POSTGRES_USER: toolkit
        POSTGRES_PASSWORD: password
    volumes: ["postgres_data:/var/lib/postgresql/data"]
    ports: ["5432:5432"]

redis:
    image: redis:7.2-alpine
    command: redis-server --appendonly yes
    volumes: ["redis_data:/data"]
    ports: ["6379:6379"]

minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    environment:
        MINIO_ROOT_USER: minioadmin
        MINIO_ROOT_PASSWORD: minioadmin
    ports: ["9000:9000", "9001:9001"]
    volumes: ["minio_data:/data"]

frontend:
    build: ./frontend
    ports: ["3000:3000"]
    environment:
        REACT_APP_API_URL: http://localhost:8000

volumes:
    postgres_data:
    redis_data:
    minio_data:

```

22.2 Kubernetes (Production)

Services deployed:

Service	Replicas	Resource Limits
api (FastAPI + Uvicorn)	3	1 CPU, 1GB RAM

Service	Replicas	Resource Limits
worker - assessment (Celery)	5 (auto-scaled)	2 CPU, 4GB RAM
worker - webhook (Celery)	2	0.5 CPU, 512MB RAM
flower (Celery monitor)	1	0.5 CPU, 512MB RAM
postgres (StatefulSet)	1 primary + 1 read replica	2 CPU, 8GB RAM
redis	1	1 CPU, 2GB RAM
frontend (Nginx)	2	0.5 CPU, 256MB RAM

Ingress: Nginx Ingress Controller with TLS termination (cert-manager + Let's Encrypt or Government CA cert).

Auto-scaling (HPA for assessment workers):

```
# k8s/worker-hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: worker-assessment-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: worker-assessment
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: External
    external:
      metric:
        name: celery_queue_length
        selector:
          matchLabels:
            queue: assessment
      target:
        type: AverageValue
        averageValue: "10" # Scale up if queue > 10 tasks per worker
```

23. Scalability Design

Bottleneck	Solution
Large file profiling	Sampling (100K rows) for files >500MB; full scan only for PII and completeness
100 concurrent assessments	Celery workers auto-scale via HPA; Redis handles queue bursts
Database write throughput	Async SQLAlchemy; connection pooling (pool_size=20, max_overflow=10)
Report generation (PDF)	WeasyPrint is CPU-heavy; isolated to worker thread; soft time limit = 60s

Bottleneck	Solution
S3 throughput	Multipart upload for files >100MB; pre-signed URLs for download avoid proxying

Processing time targets (from PRD):

Dataset Size	Target	Approach
≤100MB	< 60s	Full profiling
100MB–1GB	< 180s	Sampled profiling
1GB–5GB	< 300s	Sampled profiling + chunked read

24. Monitoring & Observability

Tool	Purpose	Access
Flower	Celery task queue monitoring (active tasks, worker status, failure rates)	Internal only
Prometheus	Scrapes FastAPI metrics (request count, latency, error rate) + Celery metrics	Internal
Grafana	Dashboard for Prometheus metrics	Internal
structlog	Structured JSON logs from all components → log aggregation	All services
PostgreSQL pg_stat_activity	Query performance monitoring	DBA access

Key metrics to alert on:

- Assessment failure rate > 5%
- Queue depth > 50 (assessment queue)
- p95 processing time > 3 minutes
- Webhook failure rate > 10%
- API error rate (5xx) > 1%

25. Domain Scorer Configuration (YAML-driven)

Scoring criteria for each domain are stored in `config/domain_criteria.yaml` rather than

hardcoded in Python. This allows the scoring rules to be updated when MIDAS 2.0 finalises post-Delphi without code changes.

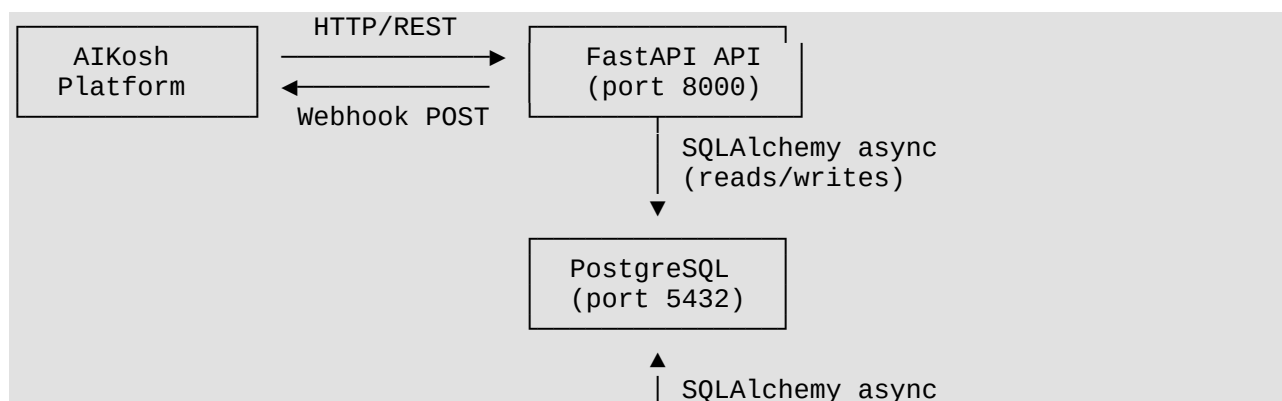
```
# config/domain_criteria.yaml

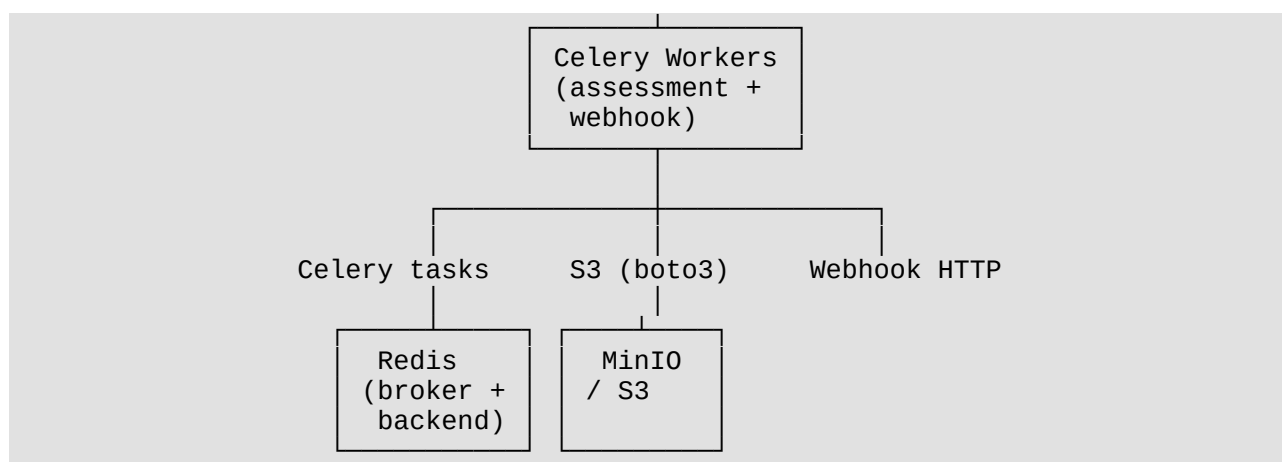
domains:
  1:
    name: "Annotation / Labelling Reliability"
    weight: 1.0
    criteria:
      score_0: "No annotation present or no information provided"
      score_1: "Annotations exist but no annotator qualification, no protocol,
no IRR metric"
      score_2: "Annotators qualified, protocol documented, but no IRR metric or
IRR < 0.6 Kappa"
      score_3: "Qualified annotators, documented protocol, IRR ≥ 0.6,
adjudication process described"
      score_4: "All of Score 3 + IRR ≥ 0.8, ≥ 2 independent annotators, gold
standard reference"
      thresholds:
        irr_adequate: 0.6
        irr_exemplary: 0.8
      inferred: true
      inferred_note: "Kappa thresholds from health informatics literature; not
explicitly stated in MIDAS 2.0 public docs"

  7:
    name: "Privacy & Identifiability"
    weight: 1.0
    criteria:
      score_0: "Direct identifiers present"
      score_1: "Direct identifiers removed but quasi-identifiers at high
granularity"
      score_2: "Quasi-identifiers generalised; no formal de-identification
documented"
      score_3: "Formal de-identification applied, documented, no direct
identifiers"
      score_4: "All of Score 3 + Differential Privacy with documented epsilon OR
k-anonymity ≥ 5"
      thresholds:
        k_anonymity_min: 5
      inferred: false

# ... all 15 domains
```

26. Inter-Service Communication Map





27. Key Design Decisions & Rationale

Decision	Chosen Approach	Why
Rule-based scoring (no ML)	Pure Python deterministic rules loaded from YAML	Scores must be auditable and reproducible; no black box acceptable in government/research context
Async job processing	Celery + Redis	Assessment can take 3 min for 1GB files; synchronous processing would timeout; Celery gives retry, monitoring, and scaling
Two separate Celery queues	assessment + webhook	Prevents a slow assessment job from blocking lightweight webhook delivery
YAML-driven scoring criteria	config/ domain_criteria.yaml	MIDAS 2.0 is not finalised; criteria may change post-Delphi review; YAML allows updates without redeploy
Dataset file deletion post-assessment	Delete after report generation	Health research datasets may contain PII; minimal retention reduces risk
Pre-signed S3 URLs for reports	Time-limited (24h) signed URLs	Avoids serving large files through the API server; direct client-to-S3 transfer
PostgreSQL JSONB for evidence/gaps	JSONB columns in domain_scores	Domain evidence is variable-length arrays; JSONB allows querying without rigid schema
Audit log append-only rule	PostgreSQL CREATE RULE no_delete_audit	Audit logs must be tamper-evident for government accountability
Equal domain weighting	1/15 per domain	MIDAS 2.0 public documentation does not specify

Decision	Chosen Approach	Why
		differential weights
Confidence level per domain	High / Medium / Low field	Distinguishes file-based scoring from metadata-form-reliant scoring; critical for report credibility

28. Dependency Versions (Pinned)

```
# requirements.txt (backend)
fastapi==0.111.1
uvicorn[standard]==0.29.0
celery[redis]==5.4.0
redis==5.0.7
sqlalchemy[asyncio]==2.0.30
asynpg==0.29.0
alembic==1.13.1
pydantic==2.7.1
pydantic-settings==2.3.0
pandas==2.2.2
numpy==1.26.4
ydata-profiling==4.9.0
pyarrow==16.1.0
openpyxl==3.1.4
chardet==5.2.0
boto3==1.34.131
jinja2==3.1.4
weasyprint==62.3
httpx==0.27.0
structlog==24.2.0
python-multipart==0.0.9
passlib==1.7.4
pip-audit==2.7.3

# requirements-dev.txt
pytest==8.2.2
pytest-asyncio==0.23.7
pytest-cov==5.0.0
httpx==0.27.0 # for TestClient
factory-boy==3.3.0
```

End of Document

Document History

Version	Date	Author	Notes
1.0	June 18, 2026	—	Initial TDD based on PRD v1.0